

A Digital Bit-Reconfigurable Versatile Compute-In-Memory Macro for Machine Learning Acceleration

Xin Zhang¹, Graduate Student Member, IEEE, Yuncheng Lu¹, Graduate Student Member, IEEE, Bo Wang¹, Senior Member, IEEE, and Tony Tae-Hyoung Kim¹, Senior Member, IEEE

Abstract—This brief proposes a digital versatile SRAM-based computing-in-memory (CIM) macro with reconfigurable precision from 1-bit to 16-bit and programmable mathematical functions, including addition and multiplication. The proposed CIM macro supports 1~16-bit weight-stationary addition (WSA) and operands-stationary addition (OSA), and 1~8-bit bit-serial multiplication (BSM). The proposed versatile CIM macro accelerates various machine learning algorithms such as convolutional neural networks (CNNs) and self-organizing maps (SOMs). A test chip was fabricated in 65nm CMOS technology and achieved an energy efficiency of up to 40.7 TOPS/W for WSA (1-bit), 39.4TOPS/W for OSA (1-bit), and 84.1 TOPS/W for BSM (1-bit).

Index Terms—Computing-in-memory, addition, multiplication, on-chip training, SRAM, machine learning.

I. INTRODUCTION

WITH the explosion of artificial intelligence (AI) technology, various ML algorithms are highlighted for accomplishing complicated missions, such as CNN for speech recognition and autonomous vehicles and SOM for processing unlabelled data and analyzing high-dimensional data [1], [2], [3]. However, CNN and SOM consume high power and suffer from large processing delays in the traditional Von Neumann architectures [4], [5], [6]. Specifically, the intensive data movement and computation lead to unsatisfied performance from the “memory wall” between processing elements and storage units [7].

Alternatively, in-memory computing has been proposed as one of the most promising approaches to break the “memory wall” and accelerate processing with low power consumption. Whereas most of the reported CIM macros with a specific structure for the computation of multiply-and-accumulate

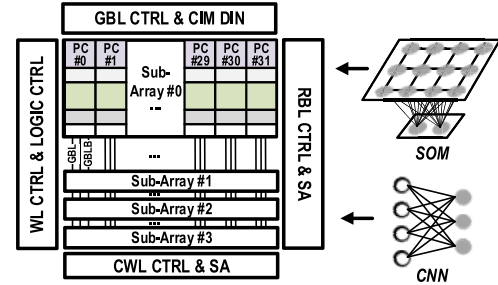


Fig. 1. Overall architecture of the proposed CIM macro.

(MAC) only aim at one type of algorithm, CNN, and are inapplicable to other algorithms [8], [9]. In [10], [11], the authors proposed analog-based CIM macros only for CNN acceleration. Unlike the analog-based CIM in [8], [9], [10], [11], the work in [12] introduced a digital CIM macro with reconfigurable 1 to 16-bit precision. A digital processing unit comprising an SRAM bit-cell, XNOR, MUX, and a full adder is proposed for MAC computation. However, the digital processing unit requires a larger area compared with the analog-based counterpart. Though the work in [12] supports reconfigurable precision, it is limited to only CNN processing. Multi-functional digital-based CIM macros were reported in [13]. However, the energy efficiency is not optimal.

To address the above issues, we proposed a digital 1~16-bit multi-functional CIM macro that applies to various machine learning algorithms.

II. PROPOSED MULTI-FUNCTIONAL CIM MACRO

A. Overall Architecture and Supported Algorithms

The overall architecture of the proposed CIM macro is shown in Fig. 1. The CIM macro consists of three parts: 1) four 37×32 SRAM sub-arrays, 2) peripheral circuits for row/column access, and 3) a CIM input circuit. Each column in the sub-array forms a processing column (PC), realizing 32 PCs in each sub-array. The four PCs in the same column position in the four sub-arrays are connected by one global bit-line (GBL/GBLB). Input data is applied to and shared by the four PCs from the GBL. The array could be adaptive to different precision from 1 to 16-bit for weight-stationary addition (WSA) and operands-stationary addition (OSA), which enables the CIM macro to accelerate the SOM

Manuscript received 12 February 2023; revised 7 March 2023; accepted 7 March 2023. Date of publication 14 March 2023; date of current version 12 May 2023. This work was supported by the RIE2020 ASTAR AME IAF-ICP under Grant I1801E0030. This brief was recommended by Associate Editor E. Yao. (Xin Zhang and Yuncheng Lu contributed equally to this work.) (Corresponding author: Tony Tae-Hyoung Kim.)

Xin Zhang, Yuncheng Lu, and Tony Tae-Hyoung Kim are with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore (e-mail: xin010@e.ntu.edu.sg; yuncheng001@e.ntu.edu.sg; thkim@ntu.edu.sg).

Bo Wang is with the Information Systems Technology and Design, Singapore University of Technology and Design, Singapore (e-mail: bo_wang@sutd.edu.sg).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TCSII.2023.3257058>.

Digital Object Identifier 10.1109/TCSII.2023.3257058

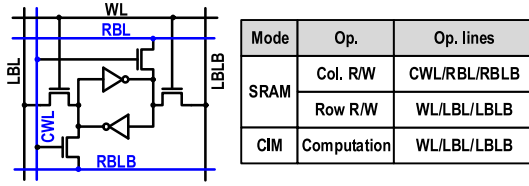


Fig. 2. Structure of 8T SRAM bit-cell.

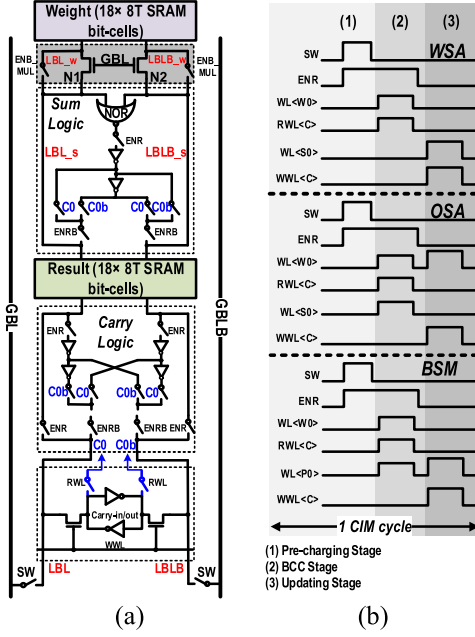


Fig. 3. (a) Structure of the PC, and (b) the waveforms of control signal in WSA, OSA, and BSM.

in both recall and weight updating phases. Besides, 1 to 8-bit for bit-serial multiplication (BSM) is also realized for accelerating the MAC operation in CNN. Since WSA, OSA, and BSM are the major and repetitive operations in both SOM and CNN, the proposed CIM macro can be reconfigured to accelerate the above applications.

B. 8T Bit-Cell and Processing Column

Fig. 2 shows the 8T SRAM bit-cell. Two additional transistors are added to the conventional 6T SRAM bit-cell for the horizontal read operation, with which the column-wise stored multi-bit data could be read out word by word. Column word-line (CWL) and row bit-line (RBL/RBLB) are employed for horizontal read-out. Word-line (WL) and local bit-lines (LBL/LBLB) are utilized for both standard SRAM row read/write and CIM processing in each PC.

Each sub-array contains thirty-two PCs. Fig. 3(a) illustrates the structure of unit PC, which consists of two groups of eighteen bit-cells for weight and computation result storage, two NMOS devices (N1 & N2) for function-select (WSA/OSA/BSM), Sum Logic, Carry Logic, and one bit-cell for carry-in/out storage. In each PC, three main stages including 1) pre-charging stage, 2) bit-cell computing (BCC) stage, and 3) updating stage are conducted sequentially in one CIM operation cycle whose waveforms of the control signal are

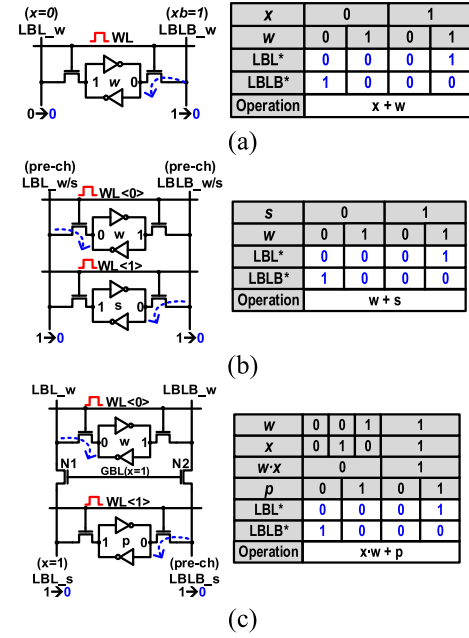


Fig. 4. (a) BCC stage and the LBL/LBLB final states in WSA, (b) in OSA, and (c) in BSM.

shown in Fig. 3(b). The first stage is for pre-charging from GBL/GBLB to LBL/LBLB. In the following BCC stage, relevant WL(s) of bit-cell(s) are activated for computing, and LBL/LBLB shows the relative final states. Lastly, in the result updating stage, the obtained sum and carry-out results are updated and stored in the specified bit-cells in the same PC. ENR is the control signal for distinguishing the period of three stages. Note that when ENR is off, LBL/LBLB is divided into three parts (LBL/LBL_s/LBL_w and LBLB/LBLB_s/LBLB_w) during the updating stage for sum and carry-out writing at the same time. When the ENR is on, the three parts of LBL/LBLB are connected and show the same states in the pre-charging and BCC stages. ENB_MUL is off in BSM, and on in OSA and WSA operations.

C. Bit-Cell Computing Stage and Results Updating Stage

The BCC stage and updating stage are conducted following the pre-charging stage. Fig. 4(a) illustrates the BCC stage in the WSA where the input x/x_b (pre-charge for “1”, ground for “0”) is loaded to LBL/LBLB from GBL/GBLB bit-serially. The 1-bit weight is stored in the bit-cell. With the WL of the weight bit-cell activated, either LBL or LBLB will be discharged to “0” or kept unchanged, depending on the stored weight. The storing data is safe when the bit-cell is involved in the WSA operation. LBL or LBLB is pre-charged to VDD representing the input “1”, instead of being connected to a strongly driven “1”, which cannot be stronger than the cross-coupled inverters. Besides, the WL voltage is lowered to guarantee that the cross-coupled inverters are stronger than the access transistors, making the stored data secure during WSA operations. The possible final states of the LBL& LBLB are shown in Fig. 4(a) table and are utilized later to calculate the full-add of “ $x + w$ ” in the next step.

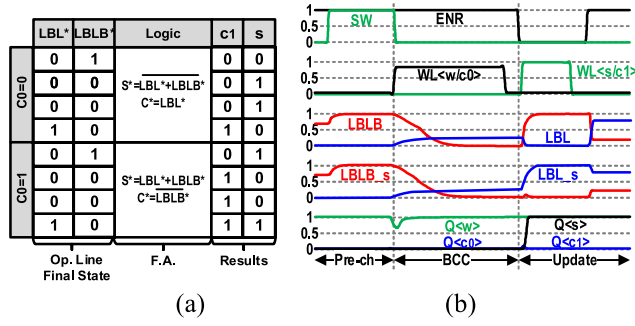


Fig. 5. (a) Full-add logic from LBL&LBLE final states, and (b) waveforms of operation lines in WSA simulation.

Fig. 4(b) shows the BCC stage in OSA, where both operands (weight “w” and partial sum “s”) are stored in the memory bit-cells and loaded simultaneously. Unlike WSA, LBL/LBLB are pre-charged by GBL/GBLB at first. Then, if there is “0” stored on either side, LBL/LBLB will be discharged accordingly when the WLs are activated. The possible final states of the LBL and LBLB are shown in Fig. 4(b) table and utilized to calculate the full-add of “w + s” later. In the BCC stage of BSM, “x·w+p” is calculated, where x is input, w is stored data, and p is a previous partial sum. w and p are pre-stored in the bit cells in the same PC. Two NMOS devices are inserted for function-select at LBL/LBLB whose gates are controlled by input data x. During the pre-charging stage, only LBLB is pre-charged, and LBL is fed with input data x (pre-charge for “1”, ground for “0”) from GBL. Whether the initial states of LBL_w/LBLB_w are aligned with LBL/LBLB are controlled by the input data x. One case is demonstrated in Fig. 4(c). When input x is “1”, the two NMOS devices will be turned on, and the bit-cells storing w and p will participate in the multiplication. With the WLs activated, LBL/LBLB will be discharged from the w bit-cell and the p bit-cell, respectively. The final states of the LBL/LBLB with different operand combinations are shown in Fig. 4(c) table and utilized for generating results later.

After the BCC stage, the final states of the LBL/LBLB are utilized in the logic for results updating. LBL/LBLB show the same final states (LBL*/LBLB*) in the three operation cases. With the acquired standard final states, the carry-in controlled logics (Sum/Carry Logic) for updating 1-bit full-add sum and carry-out are developed as shown in Fig. 5(a). The Sum and Carry Logic circuits are realized in each column with only NOR, inverters, and switches. The waveforms of the critical signals in 1-bit WSA (x=0, w=1) are shown in Fig. 5(b) for illustration. In the results updating stage, with the pre-loaded c₀ that selects the path in Sum Logic and Carry Logic, the full-adder results, sum s, and carry-out c₁ are written into the specified bit-cells. The isolated LBL/LBLB during the results updating stage enable updating s and c₁ at the same time in one CIM cycle, and thus writing time is saved by 50% (i.e., one additional write cycle).

III. MULTI-BIT MATHEMATICAL COMPUTATION

The proposed CIM macro is reconfigurable to different precision and mathematical computations for several

applications. Here, WSA, OSA, and BSM with 2-bit precision are introduced to demonstrate the multi-cycle computation and multi-function of the CIM macro. Note that WAS, OSA, and BSM are the major operations in CNNs or SOMs.

A. Operation of 2-Bit WSA

Fig. 6(a) illustrates one example of WSA with inputs “001”/“011” and weights “010”/“011”. Note that MSB “0” is filled to the input and weight to compensate for the carry-out overflow bit. Before the computation, the weights are loaded to the bit-cells in the upper weight group, and the bit-cells in the lower result group for sum and carry are initiated with “0”. 2-bit addition needs 3 CIM cycles, and n-bit needs n+1 cycles. In the first cycle, LSB of the input is applied to LBL and LBLB from GBL and GBLB. Then WL of the bit-cell storing LSB of weight is turned on for BCC. With the Sum and Carry Logic, LSB of the sum and the carry-out are updated and stored in the same cycle. Then the second and third cycles are followed for the middle bit and the MSB addition. After three CIM cycles, the results “011” and “110” are acquired and stored in two PCs, respectively.

WSA is highly desired by distance relative algorithms, for example, SOM. With addition processed in the memory, less memory access and processing delay are realized with high parallelism of the CIM operation conducted in PCs. Subtraction could be achieved by adding the operands in two’s complements format, which is frequently employed for computing the difference between two operands.

B. Operation of 2-Bit OSA

Fig. 6(b) shows the multi-cycle computation of OSA. The two 2-bit operands are both stored in one of the bit-cells in the weight and result bit-cell group. The addition starts from the LSB. In the first cycle, two WLs of bit-cells storing LSBs are turned on. Then the pre-charged LBL and LBLB are discharged or kept unchanged depending on the stored node data. With carry-in controlled logic, full-add is completed in the PC, and the sum and carry-out are written to the relative bit-cells in the same cycle. 2-bit addition needs three cycles, and n-bit needs n+1 cycles.

OSA is highlighted for weights updating in the algorithms where the weights are tuned and updated with the pre-stored results from the previous phase. For example, for the weight updating in SOM, with the pre-stored sum results from the recall phase, weights could be on-chip tuned and updated in each PC. What’s more, when the relative WL of the lower-bit bit-cell is turned on, the right shift of the pre-stored result could also be realized in the array equal to multiply with the coefficient power of two, which is the learning rate of the on-chip training algorithm.

C. Operation of 2-Bit BSM

In BSM, 1-bit of the input is applied to LBL from GBL, but LBLB is pre-charged in each CIM cycle. Fig. 6(c) illustrates the multiplication of “01” × “10” and “11” × “11” in two PCs. Five CIM cycles are required in total, and for two n-bit vector multiplication, n²+n−1 cycles are required. In the

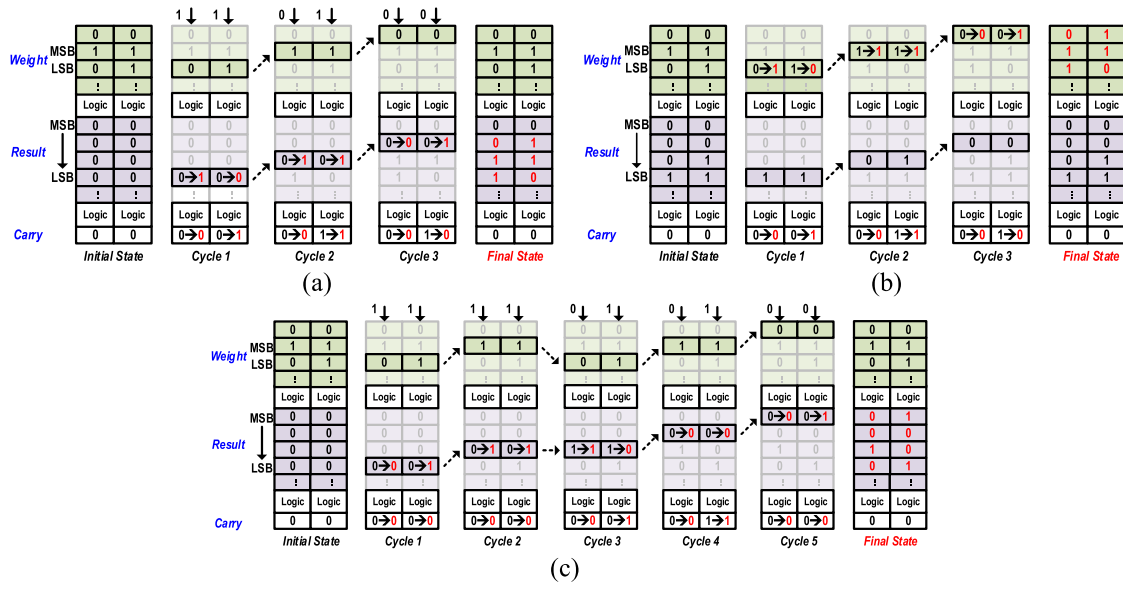


Fig. 6. (a) 3 cycles for 2-bit WSA, (b) 3 cycles of 2-bit OSA, and (c) 5 cycles for 2-bit BSM.

first cycle, LSB is applied to the PC from GBL. After turning on the WLs of the LSB bit-cells, the stored data are used for the BSM computation. Partial sum and temporary carry-out are updated later. In the next cycle, LSBs of the inputs are loaded, and the WLs of MSB are turned on. Then the multiplication is processed, and the results are added to the last cycle's partial sum and carry-in. Finally, the sum and new carry-out are obtained in the same cycle. In the third and fourth cycles, MSB of the input is multiplied by the stored 2-bit data. In the last cycle, the final overflow carry-outs are loaded to the MSB bit-cell of the sum result.

Multiplication is applicable for the CNN multiplication step (accumulation is processed out of the CIM macro). Multiply-and-accumulate (MAC) operation could be partially accelerated by the proposed CIM macro in a view of highly parallel CIM processing, with the weights of the CNN stored in the CIM array and input feature map data applied to the PCs bit-serially. However, additional post-CIM processing is required for the accumulation.

IV. EVALUATION AND MEASUREMENT RESULTS

The stability of the bit-cells needs to be investigated since two rows are activated at the same time in the OSA and BSM operations. Fig. 7 illustrates the Monte-Carlo simulation results of 1K runs at -10°C , 27°C , and 80°C , respectively. Data “1” is stored at the storing node Q of one bit-cell, and “0” in another. A lower WL voltage (0.8 V) is employed to prevent data corruption from the read disturbance.

The proposed CIM macro can be employed for addition/subtraction-based or multiplication-based ML algorithms [13]. By utilizing multiple macros, the CIM macro can be expanded for algorithms with different scales. The precision of the input and weight is programmable from 1 to 16-bit. As presented in Table I, the proposed CIM macro speeds up the multiplication and addition processing delay in CNN and SOM compared with the baselines where the CIM macro is only employed as the storage memory. As

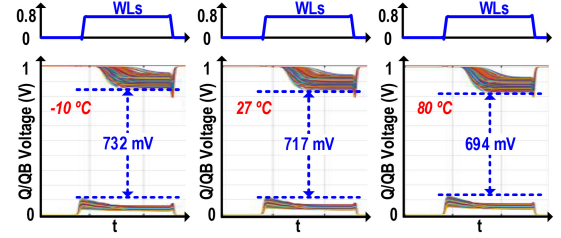


Fig. 7. Storing node voltages in Monte-Carlo simulation for two bit-cells activated at the same time in -10°C , 27°C , and 80°C .

TABLE I
COMPARISON BETWEEN THE BASELINE AND CIM

	CNN		SOM	
	Conv	FC	Recall	W Update
Input Size	6×6×6	64×1	200	/
Weight Size	3×3×6×16	16×64	4×4×32	4×4×32
Output Size	2×2×16	16×1	16	/
Input Precision	6-bit	6-bit	8-bit	8-bit ^a
Weight Precision	6-bit	6-bit	8-bit	8-bit
Macro Number	8	8	4	4
CIM Cycles	164	41	1800	9
Post-CIM cycles	128	32	128	0
Baseline Cycles	864	1024	108800	1536
Speed Up ^b	2.9×	14×	56.4×	170.7×

^a. After right-shift 1-bit with neighborhood function 2^{-1}

^b. Addition/multiplication processing are speeded up with WSA, OSA, and BSM

a demonstrator, the proposed CIM macro is used at the convolutional (Conv) layer #2 and fully connected (FC) layer #1 of LeNet-5 to claim the acceleration in multiplication processing cycles. When applying to CNN, the speed-up of multiplication in Conv and FC are 2.9× and 14×, respectively, with BSM. Besides, the recall phase and the weight update of SOM are listed to demonstrate the speed-up from the proposed CIM addition operation. In the recall phase, the difference between the inputs and the stored weights are processed simultaneously in parallel PCs, which provided a 56.4× speed-up in processing addition with WSA. In the

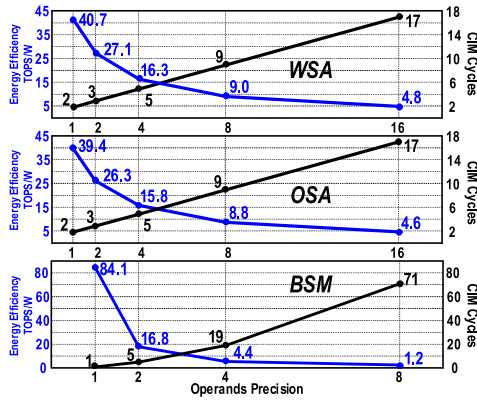


Fig. 8. Energy efficiency for WSA, OSA, and BSM with input & weight precision from 1-bit to 8/16-bit.

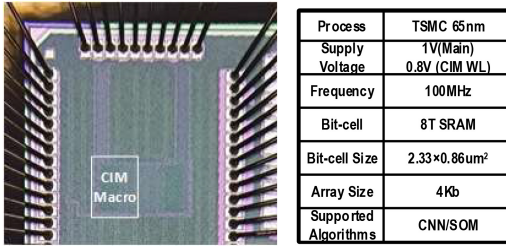


Fig. 9. Test chip die photo and chip summary.

TABLE II
COMPARISON TABLE

	ISSCC' 21 [2]	ISSCC' 22 [3]	SOVC' 22 [14]	This Work
Technology	22nm	28nm	12nm	65nm
Supply Voltage	0.72 V	0.8 V	0.72 V	0.8-1V
SRAM Size	64Kb	32Kb	8Kb	4Kb
SRAM Bit-cell	6T	6T	6T	8T
Multi-Operation Supported	No	Yes	No	Yes
W Precision	4/8/12/16b	1/4/8b	4/8b	1-16b
Multi. Energy Efficiency (TOPS/W)	24.7(8b)	27.38(8b)	30.3(8b)	84.1(1b)
Add. Energy Efficiency (TOPS/W)	-	-	-	40.7(1b)

training phase, weights are tuned and updated with pre-stored results which are calculated in the recall phase. Then without memory read-out, the weights could be updated and rewritten with OSA in PCs achieving a $170.7\times$ speed-up in the processing cycles of addition.

A test chip was fabricated in TSMC 65nm technology. Fig. 8 shows the energy efficiency and operation cycles of three CIM operations: WSA, OSA, and BSM. The input and weight precision change from 1-bit to 8/16-bit. The maximum energy efficiencies for WSA, OSA, and BSM are 40.7 TOPS/W, 39.4 TOPS/W, and 84.1 TOPS/W, respectively with 1-bit input and weight precision.

V. CONCLUSION

This brief proposed a digital SRAM-based versatile CIM macro for accelerating multiple ML algorithms. The weight

precision is reconfigurable from 1 to 16-bit for addition and 1 to 8-bit for multiplication. Aiming for speeding up different ML algorithms, the proposed CIM macro can be programmed to conduct WSA, OSA, and BSM. The test chip achieved a maximum energy efficiency of 40.7 TOPS/W, 39.4 TOPS/W, and 84.1 TOPS/W for WSA, OSA, and BSM, respectively, with 1-bit operand precision.

REFERENCES

- [1] C.-J. Jhang, C.-X. Xue, J.-M. Hung, F.-C. Chang, and M.-F. Chang, "Challenges and trends of SRAM-based computing-in-memory for AI edge devices," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 5, pp. 1773–1786, May 2021, doi: [10.1109/TCSI.2021.3064189](https://doi.org/10.1109/TCSI.2021.3064189).
- [2] Y.-D. Chih et al., "16.4 An 89TOPS/W and 16.3TOPS/mm² all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, San Francisco, CA, USA, 2021, pp. 252–254, doi: [10.1109/ISSCC42613.2021.9365766](https://doi.org/10.1109/ISSCC42613.2021.9365766).
- [3] B. Yan et al., "A 1.041-Mb/mm² 27.38-TOPS/W signed-INT8 dynamic-logic-based ADC-less SRAM compute-in-memory macro in 28nm with reconfigurable bitwise operation for AI and embedded applications," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, San Francisco, CA, USA, 2022, pp. 188–190, doi: [10.1109/ISSCC42614.2022.9731545](https://doi.org/10.1109/ISSCC42614.2022.9731545).
- [4] M. Horowitz, "1.1 computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. Dig. Techn. Papers (ISSCC)*, San Francisco, CA, USA, 2014, pp. 10–14, doi: [10.1109/ISSCC.2014.6757323](https://doi.org/10.1109/ISSCC.2014.6757323).
- [5] P.-C. Wu et al., "A 28nm 1Mb time-domain computing-in-memory 6T-SRAM macro with a 6.6ns latency, 1241GOPS and 37.01TOPS/W for 8b-MAC operations for edge-AI devices," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, San Francisco, CA, USA, 2022, pp. 1–3, doi: [10.1109/ISSCC42614.2022.9731681](https://doi.org/10.1109/ISSCC42614.2022.9731681).
- [6] J.-O. Seo, M. Seok, and S. Cho, "ARCHON: A 332.7TOPS/W 5b variation-tolerant analog CNN processor featuring analog neuronal computation unit and analog memory," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, San Francisco, CA, USA, 2022, pp. 258–260, doi: [10.1109/ISSCC42614.2022.9731654](https://doi.org/10.1109/ISSCC42614.2022.9731654).
- [7] N. Verma et al., "In-memory computing: Advances and prospects," *IEEE Solid-State Circuits Mag.*, vol. 11, no. 3, pp. 43–55, 2019, doi: [10.1109/MSSC.2019.2922889](https://doi.org/10.1109/MSSC.2019.2922889).
- [8] X. Si et al., "24.5 A twin-8T SRAM computation-in-memory macro for multiple-bit CNN-based machine learning," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, San Francisco, CA, USA, 2019, pp. 396–398, doi: [10.1109/ISSCC.2019.8662392](https://doi.org/10.1109/ISSCC.2019.8662392).
- [9] S. K. Gonugondla, M. Kang, and N. Shanbhag, "A 42pJ/decision 3.12TOPS/W robust in-memory machine learning classifier with on-chip training," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, San Francisco, CA, USA, 2018, pp. 490–492, doi: [10.1109/ISSCC.2018.8310398](https://doi.org/10.1109/ISSCC.2018.8310398).
- [10] A. Biswas and A. P. Chandrakasan, "CONV-SRAM: An energy-efficient SRAM with in-memory dot-product computation for low-power convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 54, no. 1, pp. 217–230, Jan. 2019, doi: [10.1109/JSSC.2018.2880918](https://doi.org/10.1109/JSSC.2018.2880918).
- [11] C. Yu, T. Yoo, T. T.-H. Kim, K. C. T. Chuan, and B. Kim, "A 16K current-based 8T SRAM compute-in-memory macro with decoupled read/write and 1.5bit column ADC," in *Proc. IEEE Cust. Integr. Circuits Conf. (CICC)*, Boston, MA, USA, 2020, pp. 1–4, doi: [10.1109/CICC48029.2020.9075883](https://doi.org/10.1109/CICC48029.2020.9075883).
- [12] H. Kim, Q. Chen, T. Yoo, T. T.-H. Kim, and B. Kim, "A 1-16b precision reconfigurable digital in-memory computing macro featuring column-MAC architecture and bit-serial computation," in *Proc. IEEE 45th Eur. Solid-State Circuits Conf. (ESSCIRC)*, Cracow, Poland, 2019, pp. 345–348, doi: [10.1109/ESSCIRC.2019.8902824](https://doi.org/10.1109/ESSCIRC.2019.8902824).
- [13] J. Wang et al., "14.2 A compute SRAM with bit-serial integer/floating-point operations for programmable in-memory vector acceleration," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, San Francisco, CA, USA, 2019, pp. 224–226, doi: [10.1109/ISSCC.2019.8662419](https://doi.org/10.1109/ISSCC.2019.8662419).
- [14] C.-F. Lee et al., "A 12nm 121-TOPS/W 41.6-TOPS/mm² all digital full precision SRAM-based compute-in-memory with configurable bit-width for AI edge applications," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Honolulu, HI, USA, 2022, pp. 24–25, doi: [10.1109/VLSITechnologyandCirc46769.2022.9830438](https://doi.org/10.1109/VLSITechnologyandCirc46769.2022.9830438).